Now that we have defined our hook, we are ready to define our spec and load the support file. Create a file called spec/integration/app/ledger_spec.rb with the following code:

06-integration-specs/03/expense_tracker/spec/integration/app/ledger_spec.rb
```
require_relative '../../../app/ledger'
require_relative '../../../config/sequel'
require_relative '../../support/db'

module ExpenseTracker
  RSpec.describe Ledger do
    let(:ledger) { Ledger.new }
    let(:expense) do
      {
        'payee'  => 'Starbucks',
        'amount' => 5.75,
        'date'   => '2017-06-10'
      }
    end

    describe '#record' do
      # ... contexts go here ...
    end
  end
end
```

The :ledger and :expense setup will be the same for each example, so we've used let to initialize this data.

Now, it's time to spell out the behavior we want in the first example. We want to tell the ledger to save the expense, and then actually read the database from disk and make sure the expense really got saved:

06-integration-specs/03/expense_tracker/spec/integration/app/ledger_spec.rb
```
context 'with a valid expense' do
  it 'successfully saves the expense in the DB' do
    result = ledger.record(expense)

    expect(result).to be_success
    expect(DB[:expenses].all).to match [a_hash_including(
      id: result.expense_id,
      payee: 'Starbucks',
      amount: 5.75,
      date: Date.iso8601('2017-06-10')
    )]
  end
end
```

We're using a couple of new matchers here. The first, be_success, simply checks that result.success? is true. This matcher is built into RSpec; you'll learn more about it in *Dynamic Predicates*, on page 194.